Section 3 — Field Rules & Constraints

What Are Field Rules and Constraints?

Basic type validation checks whether a value has the expected data type.

For example:

- name should be a string
- age should be an integer
- price should be a float

However, checking only the data type is often not enough.

For example, all of these values are integers:

- age = -10
- age = 25
- age = 500

The data type is correct, but some values are logically invalid.

Pydantic allows us to add **constraints** to individual fields.

Constraints can define rules such as:

- Minimum value
- Maximum value
- Greater than
- Less than
- Minimum string length
- Maximum string length
- Exact pattern
- Field description
- Title and metadata

The Field() function is commonly used to define these rules.

The Field() Function

Field() is used to add validation rules and metadata to a model field.

Conceptually:

Field
|
├── Numeric Constraints
| ├── gt
| ├── ge
| ├── lt
| └── le
|
├── String Constraints
| ├── min_length
| ├── max_length
| └── pattern
|
└── Metadata
  ├── title
  └── description

Numeric Constraints

Pydantic provides four important numeric comparison constraints.

gt — Greater Than

gt means:

value > limit

The value must be strictly greater than the specified limit.

Example concept:

Rule: price > 0

100 → Valid
1 → Valid
0 → Invalid
-10 → Invalid

---

ge — Greater Than or Equal To

ge means:

value >= limit

The value can be equal to the limit or greater.

Example concept:

Rule: age >= 18

18 → Valid
25 → Valid
17 → Invalid

---

lt — Less Than

lt means:

value < limit

The value must be strictly smaller than the specified limit.

Example concept:

Rule: discount < 100

50 → Valid
99 → Valid
100 → Invalid

---

le — Less Than or Equal To

le means:

value <= limit

The value may be equal to or smaller than the limit.

Example concept:

Rule: rating <= 5

4.5 → Valid
5 → Valid
6 → Invalid

Numeric Constraint Comparison

gt = Greater Than

value > limit
ge = Greater Than or Equal To

value >= limit
lt = Less Than

value < limit
le = Less Than or Equal To

value <= limit
The difference between gt and ge is whether the boundary value itself is accepted.
The same rule applies to lt and le.

String Length Constraints
Pydantic can validate the length of strings.
The two main constraints are:

- min_length
- max_length

These rules check the number of characters in a string.

---

min_length
min_length defines the minimum allowed number of characters.
Concept:
Rule:

Minimum length = 3
"AI" → Invalid
"Bob" → Valid
"Rahul" → Valid
This is useful for:

- Usernames
- Passwords
- Names
- Product codes
- Titles

---

max_length
max_length defines the maximum allowed number of characters.
Concept:
Rule:

Maximum length = 10
"Rahul" → Valid
"Ankan123" → Valid
"VeryLongUsername" → Invalid
This is useful when databases, APIs, or business rules impose length limits.

Combining Minimum and Maximum Length

Both rules can be used together.

Concept:

Username Length

Minimum: 3

Maximum: 20

Validation flow:

Input String

↓

Length < 3?

↓

Yes → Error

No

↓

Length > 20?

↓

Yes → Error

No

↓

Valid

Pattern Validation

Sometimes string length is not enough.

A value may need to follow a specific format.

Examples include:

- Employee ID
- Product code
- Postal code
- Phone number
- Username format
- Registration number

Pydantic supports pattern validation.

A pattern defines the expected structure of a string.

For example, suppose an employee ID must follow this format:

EMP-1234

The rule may require:

EMP-

+

exactly four digits

Valid:

EMP-1234

EMP-5678

Invalid:

EMP1234

EMP-12
ABC-1234
Pattern validation is generally implemented using regular expressions.

Field Description
A field can contain descriptive metadata.
A description explains the purpose of the field.
Descriptions do not normally change whether a value is accepted or rejected.
They help:

- Developers understand the model
- API documentation become clearer
- Generated JSON Schema contain useful information
- Teams understand field requirements

For example, a price field description may explain:
Product price in Indian Rupees
This description documents the field but does not itself create a validation rule.

Field Title
A field may also have a human-readable title.
For example:
Internal Field Name:
product_name

Title:
Product Name
Titles and descriptions are useful for documentation and schema generation.

Multiple Constraints on One Field
A single field can have multiple rules.
For example, an age field may require:
Minimum Age: 18
Maximum Age: 100
A username may require:
Minimum Length: 3
Maximum Length: 20
Pattern: letters, numbers, and underscore only
The data must satisfy all applicable rules.
Concept:
Incoming Value
↓
Type Check
↓
Rule 1
↓
Rule 2
↓
Rule 3
↓

Accepted
If any required rule fails:
Rule Failed
↓
ValidationError

Important Points

1. Type validation alone is not enough for many real-world applications.
2. Constraints define additional rules for individual fields.
3. Field() is commonly used to define validation rules and metadata.
4. gt means greater than.
5. ge means greater than or equal to.
6. lt means less than.
7. le means less than or equal to.
8. min_length defines the minimum string length.
9. max_length defines the maximum string length.
10. pattern checks whether a string follows a required format.
11. Multiple constraints can be applied to one field.
12. title and description provide metadata.
13. Metadata improves documentation but does not necessarily validate values.
14. Invalid constraint values raise validation errors.
15. Constraints help keep business rules close to the data model.

Syntax

General Field Syntax

```
field_name:
   expected_type

   Field(
      validation rules,
      metadata
   )
```

Numeric Constraint Syntax

```
Field(
   gt = minimum exclusive value
)
Field(
   ge = minimum inclusive value
)
Field(
   lt = maximum exclusive value
)
Field(
   le = maximum inclusive value
)
```

String Constraint Syntax

```
Field(
   min_length = minimum characters,
   max_length = maximum characters
)
```

Pattern Syntax

```
Field(
   pattern = required regular expression
)
```

Metadata Syntax

```
Field(
   title = human-readable field title,
   description = explanation of the field
)
```

Combined Validation Structure

```
Model
|
├── Field 1
|     ├── Type
|     ├── Minimum Rule
|     └── Maximum Rule
|
├── Field 2
|     ├── Type
|     ├── Length Rule
|     └── Pattern Rule
|
└── Field 3
      ├── Type
      ├── Title
      └── Description
```